

Implementing Basic Features of the Dashboard

Learn :

1. How to collect security logs (system logs, network logs).

Collecting Security logs :

- **System logs :** Use journalctl (for systemd) or read logs from /var/log/
- **Network logs :** Capture packets using tcpdump or analyze network activity with iptables logs.

2. How to visualize data using graphs/charts.

Use **Chart.js** in the frontend to plot logs as bar charts (e.g., error levels, timestamps).

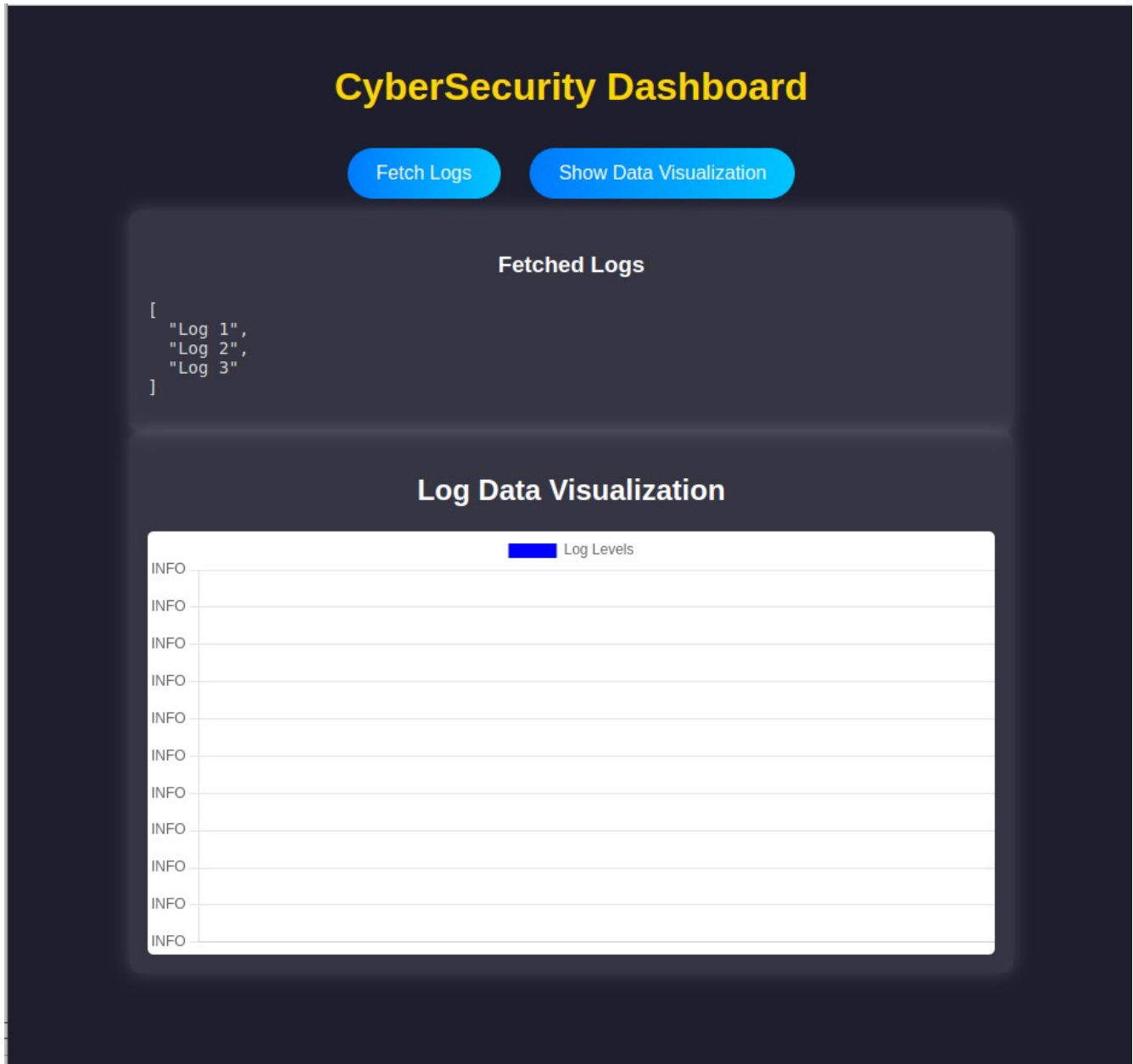
Task :

1. Write a script to collect system logs.

```
backend > app.py > get_logs
1  from flask import Flask, jsonify
2  from flask_cors import CORS
3
4  app = Flask(__name__)
5  CORS(app)
6
7  @app.route('/logs')
8  def get_logs():
9      logs = ["Log 1", "Log 2", "Log 3"]
10     return jsonify(logs)
11
12 if __name__ == "__main__":
13     app.run(debug=True)
14
```

App.py script in the backend part will be responsible for collecting system logs and serving them to the frontend.

2. Create a basic frontend for the dashboard with placeholders for data visualization.



In this dashboard :

For security logs : The Flask API (`app.py`) fetches logs from system/network sources and returns JSON data to the frontend.

To visualize data : The `fetchLogs()` function processes logs, extracts timestamps and log levels, and visualizes them dynamically using `Chart.js`

Working :

Backend (`app.py`)

- Collects system logs (from `/var/log/syslog` on Linux).
- Serves logs via a Flask API (`/logs`).

Frontend (`index.html`)

- Fetches logs using JavaScript when the user clicks "Fetch Logs".
- Displays logs in a formatted way.
- Uses **Chart.js** to visualize log levels (INFO, WARNING, ERROR) in a bar chart.
- Toggles log and chart visibility with buttons.

🔗 Final Flow:

Backend collects logs → API serves logs → Frontend displays & visualizes logs.

~Task 4 Week 5

~March 5, 2025